Python (Programming Language)

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. It was created by Guido van Rossum and first released in 1991 at the National Research Institute for Mathematics and Computer Science in the Netherlands. Python source code is now available under the GNU General Public License (GPL). It is available on the official website of Python https://www.python.org.

Python programs are executed in two different modes

- Interactive Mode Programming
- Script Mode Programming

Python Features:

- Easy to Learn and Use
- Expressive Language
- Interpreted Language
- Cross-platform Language
- Free and Open Source
- Object-Oriented Language
- Large Standard Library
- GUI Programming Support
- Integrated

Python Variables

Variable is a name which is used to refer memory location. Variable also known as identifier and used to hold value

Declaring Variable and Assigning Values

Python does not bound us to declare variable before using in the application. It allows us to create variable at required time. We don't need to declare explicitly variable in Python. When we assign any value to the variable that variable is declared automatically. The equal (=) operator is used to assign value to a variable.

```
a=30
b=”Tiger”
x=y=z=50
```

Python Indentation

Python uses indentation to indicate a block of code.

Python Comments

Comments can be used to explain Python code.
Comments can be used to make the code more readable.
Comments can be used to prevent execution when testing code.

Single line comments

```
# This is a comment
```

Multiline line comments

```
"""
This is a comment
written in
more than just one line
"""
```

Tokens:

Token is the smallest unit inside the given program.

There are following tokens in Python:

- Keywords.
- Identifiers.
- Literals.
- Operators.

Python Keywords:

Python Keywords are special reserved words which convey a special meaning to the compiler/interpreter. Each keyword have a special meaning and a specific operation. These keywords can't be used as variable. Following is the List of Python Keywords.

True	False	None	and	as
asset	def	class	continue	break
else	finally	elif	del	except
global	for	if	from	import
raise	try	or	return	pass
nonlocal	in	not	is	lambda

Identifier Naming

Variables are the example of identifiers. An Identifier is used to identify the literals used in the program. The rules to name an identifier are given below.

- The first character of the variable must be an alphabet or underscore.
- All the characters except the first character may be an alphabet of lower-case(a-z), upper-case (A-Z), underscore or digit (0-9).
- Identifier name must not contain any white-space, or special character (!, @, #, %, ^, &, *).
- Identifier name must not be keyword defined in the language.
- Identifier names are case sensitive.

Python Operators

The operator can be defined as a symbol which is responsible for a particular operation between two operands. Python operators are.

- Arithmetic Operators:
  - Addition: +
  - Subtraction: -
  - Multiplication: *
  - Division: /
  - Floor Division: // (returns the quotient without the remainder)
  - Modulo: % (returns the remainder)
  - Exponentiation: ** (raises a number to the power of another number)

- Assignment Operators:
  - Assignment: =
  - Addition assignment: += (e.g., x += 2 is equivalent to x = x + 2)
  - Subtraction assignment: -=
  - Multiplication assignment: *=
  - Division assignment: /=
  - Floor division assignment: //=
  - Modulo assignment: %=
  - Exponentiation assignment: **=

- Comparison Operators:
  - Equal to: ==
  - Not equal to: !=
  - Greater than: >
  - Less than: <
  - Greater than or equal to: >=
  - Less than or equal to: <=

- Logical Operators:
  - Logical AND: and
  - Logical OR: or
  - Logical NOT: not

- Bitwise Operators:
  - Bitwise AND: &
  - Bitwise OR: |
  - Bitwise XOR: ^
  - Bitwise NOT: ~
  - Left shift: <<
  - Right shift: >>

- Membership Operators:
  - in: Evaluates to True if a value is found in a sequence.
  - not in: Evaluates to True if a value is not found in a sequence.

- Identity Operators:
  - is: Evaluates to True if two variables refer to the same object.
  - is not: Evaluates to True if two variables do not refer to the same object.

Standard data types:

A variable can hold different types of values. For example, a student's name must be stored as a string whereas its roll no must be stored as an integer.

Python provides various standard data types that define the storage method on each of them. The data types defined in Python are given below.

- Numbers - int float,complex
- String - str
- Sequence Type - List , tupe , range
- Mapping – dictionary
- Set type – set
- Boolean – bool
- None type – None

Note: **Python also allows you to create custom data types using classes.**

List

The list can contain data of different types. The items stored in the list are separated with a comma (,) and enclosed within square brackets [].

We can use slice [:] operators to access the data of the list. The concatenation operator (+) and repetition operator (*) works with the list in the same way as they were working with the strings.

- L = [3,'Vaishali',3.5,'Jaipur']
- print (L[3:]);
- print (L[0:2]);
- print (l * 3);

Tuples:

Tuple is another form of collection where different type of data can be stored. A tuple is similar to the list in many ways. Like lists, tuples also contain the collection of the items of different data types. The items of the tuple are separated with a comma (,) and enclosed in parentheses ().

- T = (3,'Vaishali',3.5,'Jaipur')
- print (T[3:]);
- print (T[0:2]);
- print (T * 3);

Dictionary:

Dictionary is a collection which works on a key-value pair. It works like an associated array where no two keys can be same. Dictionaries are enclosed by curly braces {} and values can be retrieved by square bracket[].

- d = {1:'Amit', 2:'Sumit', 3:'Raj', 4:'Rohan'};
- print("1st name is "+d[1]);
- print (d.keys());
- print (d.values());

Python Conditions and If statements

Python, conditions and if statements allow you to control the flow of your program based on certain conditions. They enable you to execute different blocks of code based on whether a condition is true or false.

If Statements: The if statement is used to execute a block of code only if a certain condition is true. You can also include an optional else clause to execute a different block of code if the condition is false

Additionally, you can use the elif keyword (short for "else if") to add multiple conditions and code blocks:

```
x = 5

if x > 0:
   print("x is positive")
elif x < 0:
   print("x is negative")
else:
   print("x is zero")
```

Conditions and if statements allow you to make your code more flexible and responsive by executing specific blocks of code based on different conditions. You can have nested if statements, use multiple conditions, and control the program flow based on the logic you need to implement.

```
age = 25
country = "India"

if age >= 18 and country == " India ":
   print("You are eligible to vote in the India ")
else:
   print("You are not eligible to vote in the India ")
```

Loop:

We can run a single statement or set of statements repeatedly using a loop command.

While loop

Repeats a statement or group of statements while a given condition is TRUE. It tests the condition before executing the loop body.

```
counter = 0
# Initiating the loop
while counter < 10: # giving the condition
   counter = counter + 3
         print("Python Loops")
```

Using else Statement with while Loops

```
counter = 0
while (counter < 10):
    counter = counter + 3
    print("Python Loops")
# Once the condition of while loop gives False this statement will be executed
else:
    print("Code block inside the else statement")
```

Single statement while Block

```
counter = 0
while (count < 3): print("Python Loops")
```

The for Loop

Python's for loop is designed to repeatedly execute a code block while iterating through a list, tuple, dictionary, or other iterable objects of Python. The process of traversing a sequence is known as iteration.

Loop control statements:

- Break statement: This command terminates the loop's execution and transfers the program's control to the statement next to the loop
- Continue statement: This command skips the current iteration of the loop. The statements following the continue statement are not executed once the Python interpreter reaches the continue statement.
- Pass:- The pass statement is used when a statement is syntactically necessary, but no code is to be executed

The range() Function

With the help of the range() function, we may produce a series of numbers. range(10) will produce values between 0 and 9. (10 numbers).

We can give specific start, stop, and step size values in the manner range(start, stop, step size). If the step size is not specified, it defaults to 1.